Matrix Algebra

Dr. Jyotishka Datta

About this Chapter

- Not a comprehensive survey of linear algebra.
- This will be updated/uploaded by the end of this week.
- Read chapter 4 of the “Applied Multivariate Statistics with R” book by Daniel Zelterman (link on syllabus.)

Scalars

- A scalar is a single number.
- Examples: integers, real numbers, rational numbers.
- Denoted using italic font: a, n, x.
- In R, (almost) everything is a vector.

```r
a <- 3
a
```

```
[1] 3
```

- a is a scalar but R treats it as a length-1 vector

```r
a[1]
```

```
[1] 3
```

```r
length(a)
```

```
[1] 1
```

```r
is.vector(a)
```

```
[1] TRUE
```

Vectors

- A vector is a 1-D array of numbers.
- Notation: $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$.
- Represent points in space.
- Usual notation $\mathbf{a} \in \mathbb{R}^n$.

R Example: Vectors

- We can use `c` function to create vectors in R, or use pre-defined functions like `seq`.

```
v <- seq(2,8,2)
v
```

```
[1] 2 4 6 8
```

```
length(v)
```

```
[1] 4
```

```
is.vector(v)
```

```
[1] TRUE
```

```
w <- 4:1
w
```

```
[1] 4 3 2 1
```

Vector arithmetic

- We can add two vectors $\mathbf{x}$ and $\mathbf{y}$ using the same notation `x + y`.

```
v + w
```

```
[1] 6 7 8 9
```

- The inner product of a vector is defined as $\mathbf{x}^\mathbf{T}\mathbf{x} = \sum_\mathbf{i} \mathbf{x}_\mathbf{i}^\mathbf{2}$, and can be calculated as:

```
crossprod(v)
```

```
     [,1]
[1,]  120
```

- Notice the `crossprod` function returns a 1×1 matrix, not a scalar.

Componentwise product vs `crossprod`

- Similarly, the cross-product $\mathbf{x}^{\mathbf{T}}\mathbf{y}$ can be done by:

```
crossprod(v,w)
```

```
     [,1]
[1,]   40
```

- The `*` operation for vectors performs a component-wise product of vectors:

```
v*w
```

```
[1]  8 12 12  8
```

Outer product

The outer product $\mathbf{x}\mathbf{x}^T$ is an $n \times n$ matrix. This is computed using the %o% operator:

```
v %o% v
```

```
     [,1] [,2] [,3] [,4]
[1,]    4    8   12   16
[2,]    8   16   24   32
[3,]   12   24   36   48
[4,]   16   32   48   64
```

- Notice how R prints the matrix values, bordered by a useful index notation.

```
(v %o%v)[,3]
```

```
[1] 12 24 36 48
```

Matrices

- A matrix is a 2-D array of numbers.
- Notation: $A \in \mathbb{R}^{m \times n}$.
- Example:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix}$$

- Obviously, a_{ij} or $a_{i,j}$ denotes the element on the ith row and j^{th} column.

Data matrix

- We usually express "data" in matrix language, for example, if we have p predictors on n subjects, we can denote the data as a $n \times p$ matrix $\mathbf{X}$.

$$X = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1p} \\ x_{21} & x_{22} & \dots & x_{2p} \\ \dots & \dots & \vdots & \dots \\ x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}$$

- The mean vector or centroid could be written as:

$$\bar{\mathbf{x}} = \begin{pmatrix} \bar{x}_1 \\ \bar{x}_2 \\ \vdots \\ \bar{x}_n \end{pmatrix} \in \mathbb{R}^n$$

R Example: Matrices

```r
data(mtcars)
head(mtcars)
```

```
                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

```r
dim(mtcars)
```

```
[1] 32 11
```

Matrices

- To arrange values into a matrix, we use the `matrix()` function:

```
A <- matrix(1 : 6, nrow = 2, ncol = 3)
A
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

- Individual entries can be referred to using a pair of indices. For example, the element in the second row, third column can be printed as:

```
A[2, 3]
```

```
[1] 6
```

Rows, Columns

- An entire row or column can be retrieved by specifying its index and leaving the other index empty:

```
A[2, ]
```

```
[1] 2 4 6
```

- Similarly to the way R works with vectors, you can use negative indices to exclude row or colums of a matrix:

```
A[-2,]
```

```
[1] 1 3 5
```

Matrix indices

- Note how the values 1,2,3,4,5,6 are used to create the matrix: the first two are used to fill the first column, then the next two to fill the second column, and so on.
- R allows matrices to be indexed by a single number, e.g.,

```
A[5]
```

```
[1] 5
```

- The fifth element of a is the fifth element of the vector obtained by “unrolling” the matrix, column by column.

Fill by row

- Sometimes you need to fill a matrix row by row, instead than column by column.
- You can change the default behavior as follows:

```r
A <- matrix(1 : 6, nrow = 2, ncol = 3, byrow = TRUE)
A
```

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

Matrix Transpose

- The transpose of a matrix A is denoted as A^T.
- Example:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \end{bmatrix} \Rightarrow A^T = \begin{bmatrix} a_{1,1} & a_{2,1} \\ a_{1,2} & a_{2,2} \\ a_{1,3} & a_{2,3} \end{bmatrix}$$

- You can think of transpose as mirror image across the main diagonal. Rows of A become columns of A^T and vice versa.
- Useful: $(AB)^T = B^T A^T$.
- A is called a symmetric matrix if $A^T = A$.

R Example: Matrix Transpose

```r
matrix_A <- matrix(c(1, 2, 3, 4), nrow=2, ncol=2)
print(matrix_A)
```

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
```

```r
t_A <- t(matrix_A)
print(t_A)
```

```
     [,1] [,2]
[1,]    1    2
[2,]    3    4
```

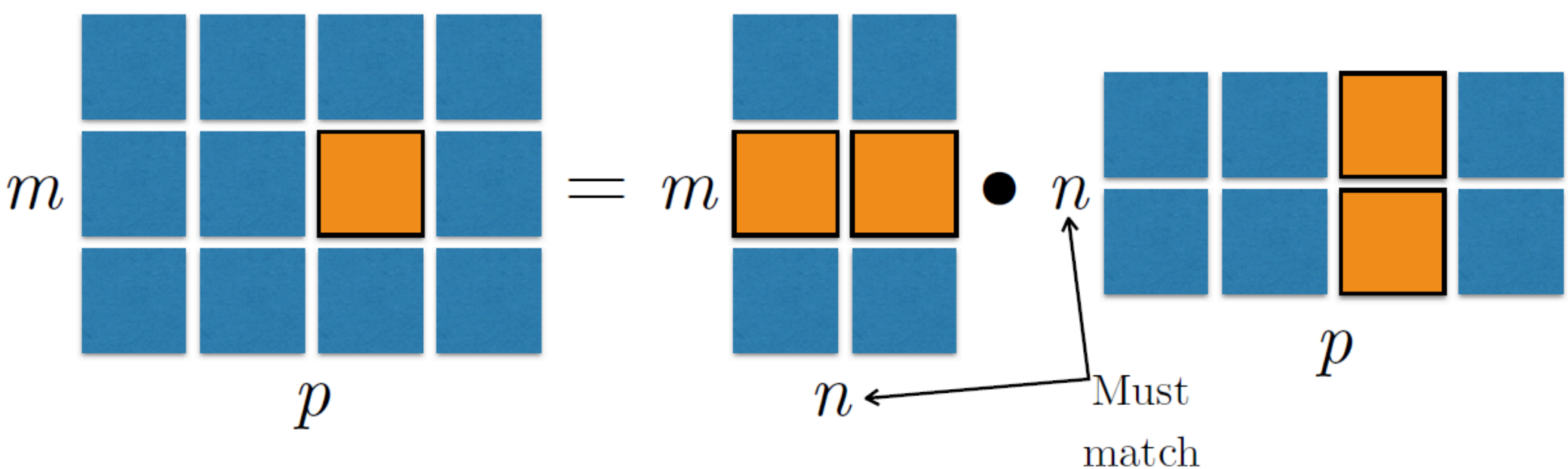
- The `t()` function takes transpose of a vector, into an $1 \times n$ matrix, so you could also do `crossprod` this way:

```r
t(v)%*%w
```

```
     [,1]
[1,]   40
```

Matrix Multiplication

- If $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, then $C = AB$ is defined.
- Each element: $c_{i,j} = \sum_k a_{i,k} b_{k,j}$.



R Example: Matrix Multiplication

- Matrix multiplication in R is performed by the operator %*%. For example,

```r
A <- matrix(c(1, 2, 3, 4), nrow=2)
B <- matrix(c(2, 0, 1, 2), nrow=2)
C <- A %*% B
print(C)
```

```
     [,1] [,2]
[1,]    2    7
[2,]    4   10
```

Multiplication

- Matrices must conform, i.e. multiplication must make sense !

```r
A <- matrix(c(1, 2, 0, 1), 2)
B <- matrix(c(1, 3, 3, 1, 4, 5), nr = 2)
A
```

```
     [,1] [,2]
[1,]    1    0
[2,]    2    1
```

```r
B
```

```
     [,1] [,2] [,3]
[1,]    1    3    4
[2,]    3    1    5
```

```r
A %*% B
```

```
     [,1] [,2] [,3]
[1,]    1    3    4
[2,]    5    7   13
```

Multiplication

- A is a 2×2 matrix and B is a 2×3 matrix, so AB can be computed but not BA!

```r
B %*% A
```

```
Error in B %*% A: non-conformable arguments
```

```r
dim(A)
```

```
[1] 2 2
```

```r
dim(B)
```

```
[1] 2 3
```

Multiplication

- With A and B defined as above the matrix product BA in not defined, but the product $B^T A$ is (B^T denotes the transpose of B).
- The transpose of a matrix can be obtained with the function `t()`.

```
t(B) %*% A
```

```
     [,1] [,2]
[1,]    7    3
[2,]    5    1
[3,]   14    5
```

Crossprod

- A much more efficient way of computing the same matrix product is via the function `crossprod()`.

```
all.equal(crossprod(B, A), t(B) %*% A)
```

```
[1] TRUE
```

- A similar function, tcrossprod(), computes the matrix product AB^T whenever the product is well defined.

Exercise (Try at home)

- Consider the matrix

```r
X <- cbind(1, seq(-1, 1, length = 11))
X
```

```
      [,1] [,2]
 [1,]    1 -1.0
 [2,]    1 -0.8
 [3,]    1 -0.6
 [4,]    1 -0.4
 [5,]    1 -0.2
 [6,]    1  0.0
 [7,]    1  0.2
 [8,]    1  0.4
 [9,]    1  0.6
[10,]    1  0.8
[11,]    1  1.0
```

Identity Matrix

- The identity matrix I_n satisfies $I_n x = x$ for any x.
- Example:

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

```r
I3 <- diag(3)
print(I3)
```

```
     [,1] [,2] [,3]
[1,]    1    0    0
[2,]    0    1    0
[3,]    0    0    1
```

Special matrices

- *Triangular* Upper triangular when $A_{ij} = 0$ for $i < j$, elements above diagonal are zero. Similarly, lower triangular when $A_{ij} = 0$ for $i > j$.
- *Diagonal and tridiagonal* Diagonal matrices have all non-diagonal entries zero ($A_{ij} = 0$ for $i \neq j$) and tridiagonal matrices have $A_{ij} = 0$ for $|i - j| > 1$.
- *Positive definite* A symmetric with $x^T A x > 0$ for all $x \neq 0$
- *Orthogonal* $A^T A = I$.

The `diag` function

- The `diag` function can be used to both build diagonal matrices as well as extract the diagonal elements of an existing matrix.

```
t(matrix(1 : 16, 4, 4))
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    5    6    7    8
[3,]    9   10   11   12
[4,]   13   14   15   16
```

```
diag(t(matrix(1 : 16, 4, 4)))
```

```
[1]  1  6 11 16
```

- Put another `diag` if you want the diagnoal elements as a diagonal matrix.

```
diag(diag(t(matrix(1 : 16, 4, 4)))
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    0    0    0
[2,]    0    6    0    0
[3,]    0    0   11    0
[4,]    0    0    0   16
```

Diagonals

- We have seen `rbind()` and `cbind()` before.

```
rbind(1:4, 2:5)
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    2    3    4    5
```

- e.g. We can use `cbind()` to construct a 3 x 3 Hilbert matrix. The (i, j) entry is $1/(i+j-1)$. (canonical examples of ill-conditioned matrices, being notoriously difficult to use in numerical computation.)

```
H3 <- 1 / cbind(1 : 3, 2 : 4, 3 : 5)
H3
```

```
          [,1]      [,2]      [,3]
[1,] 1.0000000 0.5000000 0.3333333
```

Diagonals and Determinants

- The determinant of a square matrix can be obtained with `det()`.

```
det(H3)
```

```
[1] 0.000462963
```

- For a diagonal matrix determinant is the product of the diagonal elements.

```
A = diag(1:4)
A
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    0    0    0
[2,]    0    2    0    0
[3,]    0    0    3    0
[4,]    0    0    0    4
```

```
det(A)
```

Triangular

- The functions `lower.tri()` and `upper.tri()` can be used to obtain the lower and upper triangular parts of matrices.
- The output of the functions is a matrix of logical elements, with TRUE representing the relevant triangular elements. For example,

```r
lower.tri(H3)
```

```
      [,1]  [,2]  [,3]
[1,] FALSE FALSE FALSE
[2,]  TRUE FALSE FALSE
[3,]  TRUE  TRUE FALSE
```

Triangular

- A typical use of these functions is to set the upper or lower triangular part of a matrix to zero, thus constructing a triangular matrix.

```
Htri <- H3
Htri[lower.tri(Htri)] <- 0
Htri
```

```
     [,1]      [,2]      [,3]
[1,]    1 0.5000000 0.3333333
[2,]    0 0.3333333 0.2500000
[3,]    0 0.0000000 0.2000000
```

Eigendecomposition

An eigenvectorof a square matrix A is a nonzero vector v such that multiplication by A alters only the scale of v:

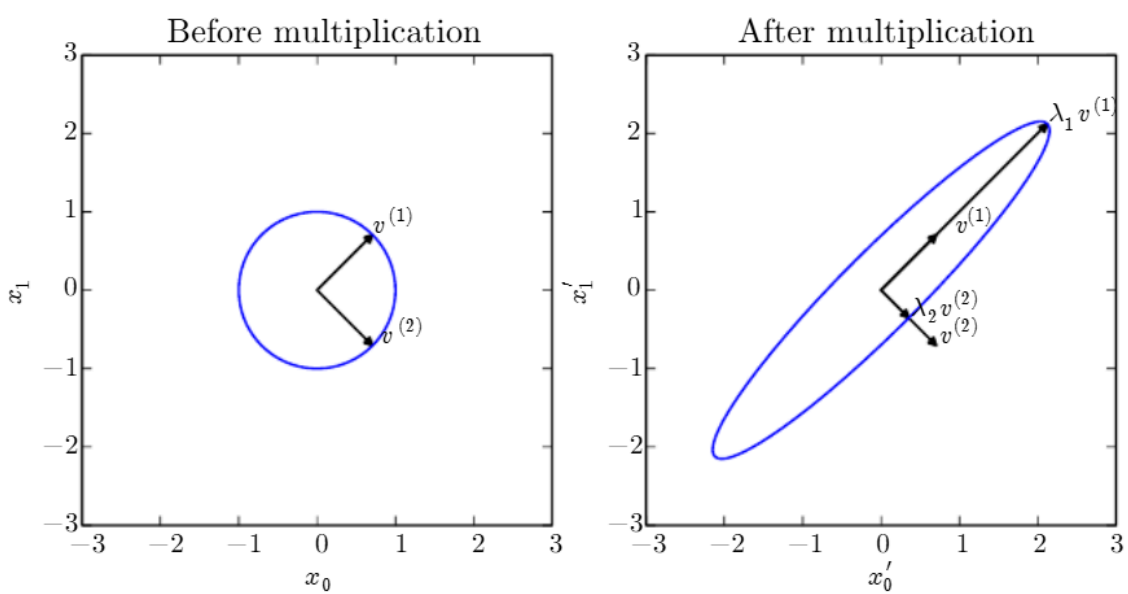
- Eigenvector v: $Av = \lambda v$.
- The scalar λ is known as the eigenvalue corresponding to this eigenvector.
- Eigenvectors are defined up to a scalar multiple, i.e. if v, λ is a solution, $2v$, $\lambda/2$ is also a solution. (In R, we take unit length and mutually orthogonal eigenvectors.)
- mutually orthogonal: $v^T v = 1, w^T w = 1, v^T w = 0.$

Eigendecomposition

- Multiplication of a vector by a scalar results in changing its length by either stretching ($\lambda > 1$), shrinking ($0 < \lambda < 1$), or reversing direction by changing its signs ($\lambda > 0$).
- The identity matrix satisfies $Ix = x$, so the identity matrix has all eigenvalues as 1 and every vector x is an eigenvector.
- What are the eigenvalues of a diagonal matrix?

Effect of eigenvectors

- A is a matrix with two eigenvectors $v^{(1)}$ and $v^{(2)}$ and eigenvalues λ_1 and λ_2. The figure shows multiplying an unit circle $\mathbf{u}$ with the matrix A.



Characteristic polynomial

- How do we find them? The characteristic polynomial is a polynomial in λ and the roots of the equation $p(\lambda) = 0$ determine the eigenvalues.[1]

$$p(\lambda) = \mathrm{Det}(A - \lambda I)$$

Eigen-decomposition

- Suppose that a matrix $\mathbf{A}$ has n linearly independent eigenvectors $\{\mathbf{v}^{(1)}, \ldots, \mathbf{v}^{(n)}\}$ with corresponding eigenvalues $\{\lambda_1, \ldots, \lambda_n\}$.
- We may concatenate all the eigenvectors to form a matrix $\mathbf{V}$ with one eigenvector per column: $\mathbf{V} = [\mathbf{v}^{(1)}, \ldots, \mathbf{v}^{(n)}]$.
- Likewise, we can concatenate the eigenvalues to form a vector $\boldsymbol{\lambda} = [\lambda_1, \ldots, \lambda_n]^\top$.

Eigendecomposition

- The eigen-decomposition of a “diagonalizable” $\mathbf{A}$ is then given by

$$\mathbf{A} = \mathbf{V}\,\mathrm{diag}(\boldsymbol{\lambda})\,\mathbf{V}^{-1}.$$

- We often want to decompose matrices into their eigenvalues and eigenvectors. Doing so can helpus analyze certain properties of the matrix.
- Not every matrix can be decomposed into eigenvalues and eigenvectors.

Real, symmetric matrices

- Every real symmetric matrix has a real, orthogonal eigendecomposition:

$$A = Q\Lambda Q^T$$

R Example: Eigendecomposition

```r
A <- matrix(c(3, 1, 1, 3), nrow=2)
eigen_A <- eigen(A)
print(eigen_A)
```

```
eigen() decomposition
$values
[1] 4 2

$vectors
          [,1]       [,2]
[1,] 0.7071068 -0.7071068
[2,] 0.7071068  0.7071068
```

```r
B <- matrix(c(0, 1, -1, 0), nrow=2)
eigen_B <- eigen(B)
print(eigen_B)
```

```
eigen() decomposition
$values
[1] 0+1i 0-1i

$vectors
                     [,1]                  [,2]
[1,] 0.7071068+0.0000000i 0.7071068+0.0000000i
[2,] 0.0000000-0.7071068i 0.0000000+0.7071068i
```

Positive definite matrix

- A square matrix A is said to be positive definite if for every non-zero vector x we have:

$$x^T A x > 0$$

and non-negative definite if $x^T A x \geq 0$.

- Useful property of positive definite matrices is all of their eigenvalues are real and non-negative.
- An example of p.d. matrix (or at least n.n.d) is the dispersion matrix or variance covariance matrix.

Variance-Covariance Matrix

- Measures the variance and covariance between variables.
- Notation: $S = \frac{1}{n-1}(X - \bar{X})(X - \bar{X})^T$

R Example: Variance-Covariance Matrix

```r
data_matrix <- matrix(rnorm(20), nrow=5)
cov_matrix <- cov(data_matrix)
print(cov_matrix)
```

```
            [,1]        [,2]        [,3]       [,4]
[1,]  0.86877503  1.18479779 -0.24357280 0.03324524
[2,]  1.18479779  2.50420869 -0.57016811 0.07724745
[3,] -0.24357280 -0.57016811  1.56417020 0.03600728
[4,]  0.03324524  0.07724745  0.03600728 0.03336001
```

```r
eigen(cov_matrix)$values
```

```
[1] 3.3425117 1.3539944 0.2452064 0.0288015
```

Some useful properties:

$$\text{trace}(A) = \sum_i a_{ii} = \sum_{i=1}^{n} \lambda_i.\,, \text{ and } \text{Det}(A) = \lambda_1 \cdots \lambda_i$$

```r
A
```

```
     [,1] [,2]
[1,]    3    1
[2,]    1    3
```

```r
prod(eigen(A)$values)
```

```
[1] 8
```

```r
det(A)
```

```
[1] 8
```

```r
sum(eigen(A)$values)
```

```
[1] 6
```

```r
sum(diag(A))
```

```
[1] 6
```

Singular Value Decomposition (SVD)

- Similar to eigendecomposition but more general.
- Every real matrix has an SVD: $A = UDV^T$.
- Used in dimensionality reduction, pseudoinverse computation.

R Example: SVD

```r
svd_A <- svd(A)
print(svd_A)
```

```
$d
[1] 4 2

$u
           [,1]       [,2]
[1,] -0.7071068 -0.7071068
[2,] -0.7071068  0.7071068

$v
           [,1]       [,2]
[1,] -0.7071068 -0.7071068
[2,] -0.7071068  0.7071068
```

Trace Operator

- The trace of a square matrix is the sum of its diagonal elements.
- Notation: $\mathrm{trace}(A) = \sum_i A_{i,i}$.
- Useful in matrix calculus and optimization.

```r
tr_A <- sum(diag(A))
print(tr_A)
```

```
[1] 6
```

- An useful property is:

$$\mathrm{trace}(AB) = \mathrm{trace}(BA).$$

Matrix inversion

- The inverse of a matrix can be found using `solve()`:

```
Ainv <- solve(A)
Ainv
```

```
       [,1]   [,2]
[1,]  0.375 -0.125
[2,] -0.125  0.375
```

```
all.equal(Ainv %*% A, diag(2))
```

```
[1] TRUE
```

Linear systems

- Many common problems in Statistics require solving a linear system of equations:

$$Ax = b.$$

- Examples: Least squares normal equations for linear regression: $(X^T X)\beta = X^T y$
- or the stationary distribution of a Markov chain $\pi P = \pi$, where $\sum \pi = 1$.

Linear systems

- A typical linear model can be written in matrix form as:

$$\begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1p} \\ x_{21} & x_{22} & \dots & x_{2p} \\ \dots & \dots & \dots & \dots \\ x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix} \times \begin{bmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{bmatrix}$$

Ordinary least squares

- The ordinary least squares (OLS) method is used to estimate the $p \times 1$ parameter vector $\boldsymbol{\beta}$ of the linear regression model $\mathbf{Y} = \mathbf{X}\boldsymbol{\beta}$.
- The estimator $\hat{\boldsymbol{\beta}}_{\mathrm{OLS}}$ is defined as:

$$\hat{\boldsymbol{\beta}}_{\mathrm{OLS}} = \underset{\boldsymbol{\beta} \in \mathbb{R}^p}{\operatorname{argmin}} \|\mathbf{Y} - \mathbf{X}\boldsymbol{\beta}\|^2 = \underset{\boldsymbol{\beta} \in \mathbb{R}^p}{\operatorname{argmin}} (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta})'(\mathbf{Y} -$$

Ordinary least squares

- That is, $\hat{\boldsymbol{\beta}}_{\mathrm{OLS}}$ is the choice of $\boldsymbol{\beta}$ that minimizes the squared norm of the deviation (error) between the observed values contained in $\mathbf{Y}$ and the estimated values contained in $\mathbf{X}\boldsymbol{\beta}$.

Ordinary least squares

- To derive $\hat{\boldsymbol{\beta}}_{\mathrm{OLS}}$, let $f = (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta})'(\mathbf{Y} - \mathbf{X}\boldsymbol{\beta})$. Expanding f yields:

$$
\begin{aligned}
f &= \mathbf{Y}'\mathbf{Y} - \boldsymbol{\beta}'\mathbf{X}'\mathbf{Y} - \mathbf{Y}'\mathbf{X}\boldsymbol{\beta} + \boldsymbol{\beta}'\mathbf{X}'\mathbf{X}\boldsymbol{\beta} \\
&= \mathbf{Y}'\mathbf{Y} - 2\boldsymbol{\beta}'\mathbf{X}'\mathbf{Y} + \boldsymbol{\beta}'\mathbf{X}'\mathbf{X}\boldsymbol{\beta}
\end{aligned}
$$

Ordinary least squares

- Differentiate to find the value of $\boldsymbol{\beta}$ which minimizes f:

$$\frac{\boldsymbol{\delta} f}{\boldsymbol{\delta \beta}} = -2\mathbf{X}'\mathbf{Y} + 2\mathbf{X}'\mathbf{X}\boldsymbol{\beta} = \mathbf{0} \;\rightarrow\; \mathbf{X}'\mathbf{X}\boldsymbol{\beta} = \mathbf{X}'\mathbf{Y}$$

$$\rightarrow\; \hat{\boldsymbol{\beta}}_{\mathrm{OLS}} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{Y}$$

Ordinary least squares

Verify that the estimator $\hat{\boldsymbol{\beta}}_{\mathrm{OLS}}$ is a minimum by examining the matrix of second derivatives:

$$\frac{\boldsymbol{\delta}^2 f}{\boldsymbol{\delta}\boldsymbol{\beta}^2} = 2\mathbf{X}'\mathbf{X}$$

System of Equations

- A linear system of equations can have:
  - No solution
  - Many solutions
  - Exactly one solution: this means multiplication by the matrix is an invertible function.

Linear system

- If the matrix A is a square non-singular matrix, the usual solution to the $Ax = b$ is $x = A^{-1}b$.

$$
\begin{aligned}
Ax &= b \\
\Rightarrow A^{-1}Ax &= A^{-1}b \\
\Rightarrow I_n x &= A^{-1}b \equiv x = A^{-1}b.
\end{aligned}
$$

- Directly inverting the matrix A is inefficient and could be numerically inaccurate $\Rightarrow$ Gaussian elimination!

Invertibility

- Matrix can't be inverted if…
- More rows than columns
- More columns than rows
- Redundant rows/columns ("linearly dependent", "low rank")

Invertibility

- For a square matrix,
  - invertibility is equivalent to
  - having a non-zero determinant,
  - full rank (meaning the rank equals the number of rows/columns),
  - and linearly independent columns (or rows); essentially, all these conditions imply the matrix can be inverted and vice versa.
- A square matrix with linearly dependent columns is known as singular.

Check for singular

```r
A <- matrix(c(2, 4, 1, 2), nrow = 2, byrow = TRUE)

A
```

```
     [,1] [,2]
[1,]    2    4
[2,]    1    2
```

```r
det(A)
```

```
[1] 0
```

```r
qr(A)$rank  # QR decomposition gives rank
```

```
[1] 1
```

```r
(qr(A)$rank == ncol(A))
```

```
[1] FALSE
```

Check non-singular

```r
A <- matrix(c(2, 4, 1, 3), nrow = 2, byrow = TRUE)

A
```

```
     [,1] [,2]
[1,]    2    4
[2,]    1    3
```

```r
det(A)
```

```
[1] 2
```

```r
qr(A)$rank  # QR decomposition gives rank
```

```
[1] 2
```

```r
(qr(A)$rank == ncol(A))
```

```
[1] TRUE
```

Solve linear system

- Inverse in R uses QR decomposition, that we might briefly talk about at the very end of this course (if time permits)!
- For linear systems we don't need to compute inverses. We can use Gaussian Elimination.

```r
b <- c(5, 3)
solve(A, b) # use this
```

```
[1] 1.5 0.5
```

```r
Ainv %*% b # don't use this
```

```
     [,1]
[1,]  1.5
[2,]  0.5
```

Time comparison

- Compare execution times for the different methods of solving the Least Squares problem when the number of observations is large.
- Let us start by making up our own random data: $n \times p$ design matrix X and an $n \times 1$ response vector y.
- Take $n = 5 \times 10^5$.

```
set.seed(123)
n <- 5e5; p <- 20
X <- matrix(rnorm(n * p, mean = 1 : p, sd = 10), nr = n, nc = p, byrow = TR
y <- rowSums(X) + rnorm(n)
```

OLS

- The naive approach to solve the LS problem is to compute: $\hat{\beta} = (X^T X)^{-1} X^T y$

```
system.time(bHat1 <- solve(t(X) %*% X) %*% t(X) %*% y)
```

```
user  system elapsed
0.41    0.04    0.46
```

- Not so naive: use `crossprod`.

```
system.time(bHat2 <- solve(crossprod(X), crossprod(X, y)))
```

```
user  system elapsed
0.17    0.00    0.17
```

Moore-Penrose Pseudoinverse

- Used when matrix inversion is not possible.
- Singular value decomposition allows calculation of the pseudo-inverse. Say A admits an SVD: $A = UDV^T$.
- Defined as: $A^+ = VD^+U^T$.
- Where D^+ is constructed by taking the reciprocal of the non-zero entries in D.
- Provides least-squares solutions to systems of equations.

R Example: Pseudoinverse

```r
library(MASS)
pinv_A <- ginv(A)
print(pinv_A)
```

```
     [,1] [,2]
[1,]  1.5   -2
[2,] -0.5    1
```

Correlation Matrix

- Standardized measure of association between variables.
- Notation: $R = D^{-1} S D^{-1}$

R Example: Correlation Matrix

```r
corr_matrix <- cor(data_matrix)
print(corr_matrix)
```

```
           [,1]       [,2]       [,3]      [,4]
[1,]  1.0000000  0.8032585 -0.2089455 0.1952822
[2,]  0.8032585  1.0000000 -0.2880883 0.2672611
[3,] -0.2089455 -0.2880883  1.0000000 0.1576287
[4,]  0.1952822  0.2672611  0.1576287 1.0000000
```

Standardization (Scaling and Centering)

- Subtract mean and divide by standard deviation.

R Example: Standardization

```r
scaled_data <- scale(data_matrix)
print(scaled_data)
```

```
            [,1]       [,2]       [,3]       [,4]
[1,] -0.92805965 -1.5298129  0.4575249 -1.1498318
[2,]  1.67019387  0.9668330 -0.2850211  0.2874545
[3,]  0.01180761  0.6353605  0.9593590 -0.2159134
[4,] -0.55697669 -0.4353169  0.4705307  1.5309323
[5,] -0.19696514  0.3629363 -1.6023935 -0.4526415
attr(,"scaled:center")
[1] -0.5604064  0.7723515 -0.5536423 -0.1635232
attr(,"scaled:scale")
[1] 0.9320810 1.5824692 1.2506679 0.1826472
```

More advanced topics

Inverting a matrix using Cholesky helps

- A symmetric positive definite matrix can be factorized as:

$$A = LL^T$$

```
p = 1000; A = array(rnorm(p*p), c(p,p))
A = crossprod(A)
ptm <- proc.time(); B = solve(A); proc.time()-ptm
```

```
user  system elapsed
0.83    0.00    0.83
```

```
ptm <- proc.time(); B = chol2inv(A); proc.time()-ptm
```

```
user  system elapsed
0.25    0.00    0.25
```